

# BhoomiX: AI-Assisted Cadastral Mapping Platform

## 1. Website Workflow (How to use BhoomiX)

BhoomiX is designed to streamline the process of mapping land parcels (cadastral mapping) using AI and human verification. Here is the step-by-step workflow:

### Step 1: Drone Imagery Upload

- **Action:** The user (e.g., a surveyor or administrator) goes to the "Upload Hub" by clicking the "UPLOAD TEST" button in the top navigation bar.
- **Process:**
  - The user uploads a drone image (like a GeoTIFF or high-resolution photo) of a land area.
  - The system sends this image to the backend server.
  - The AI model (simulated in this version) processes the image to automatically detect boundaries of land parcels (plots).
  - The newly detected parcels are saved in the database with a status of `ai_suggestion`.
  - The system also checks if any of these new AI suggestions physically overlap with each other or existing parcels. If they do, they are flagged as a `conflict`.

### Step 2: Surveyor Triage (Verification)

- **Action:** The user returns to the main dashboard where the map and the "Surveyor Triage" sidebar are displayed.
- **Process:**
  - The sidebar lists all the parcels that need human review, categorized into "Conflicts" (red) and "AI Suggestions" (orange).
  - The user can click "Fly To" to zoom the map exactly to where the parcel is located.
  - For each parcel, the user can:
    - **Approve (Checkmark):** Confirms the AI did a perfect job. The parcel turns green (`confirmed`) and is saved as official.
    - **Reject (X):** Discards the AI suggestion completely because it is incorrect.

### Step 3: Boundary Editing and Correction

- **Action:** If the AI suggestion is close but not quite right, the user can edit it manually.
- **Process:**
  - The user selects a parcel, and its vertices (corner points) appear on the map.
  - Using the map tools, the user clicks and drags the points to fix the boundaries.
  - Once fixed, the user clicks "Edit & Save" in the sidebar.
  - The system updates the parcel's shape in the database and changes its status to `reviewed_edited`.

### Step 4: Official Reporting and Export

- **Action:** The user can generate official documents for any verified parcel.
- **Process:**
  - The user clicks on a confirmed parcel on the map, opening a popup.
  - By clicking "Export Report", a modal opens with a printable summary containing the Parcel ID, Status, AI Confidence, and Calculated Area.
  - From here, the user can "Print Official Report" or "Download GeoJSON" to get the data file.

### Step 5: Model Feedback Loop (Continuous Improvement)

- **Action:** Admin/User clicks the "Sync Feedback" button in the top navigation bar.
- **Process:**
  - Every time a surveyor edits or rejects an AI parcel, the system silently records this "correction" in an audit log.
  - Clicking "Sync Feedback" downloads a structured dataset containing the AI's original guess and the human's final correction.
  - This dataset is used to re-train the AI model later, making it smarter for the next set of images.

## 2. Technical Stack (Under the Hood)

Here is a breakdown of the technologies powering BhoomiX and how they communicate with each other.

### A. The Frontend (What the User Sees)

- **Next.js & React:** The core framework building the website. Think of React as the building blocks (components like buttons, sidebars) and Next.js as the manager that stitches them together into web pages and handles routing (moving between pages).
- **Tailwind CSS:** A styling tool used to make the website look beautiful, modern, and dark-themed without writing messy traditional CSS.
- **MapLibre GL JS:** The interactive map engine. It is responsible for rendering the base map (roads, satellite) and drawing the colored polygons (parcels) on top of it.
- **Mapbox Draw:** A plugin attached to MapLibre that allows the user to click, drag, and draw new polygon shapes directly on the map.
- **Lucide React:** The library providing the clean, minimal icons used throughout the interface (like the checkmarks, crosshairs, and shields).

### B. The Backend (The Server and API)

- **Next.js API Routes:** Next.js doesn't just build the frontend; it also creates the "API endpoints" (the middleman). When the frontend needs to save data or run a check, it sends a message (HTTP request) to these routes (e.g., `/api/edit-parcel`).
- **TypeScript:** The programming language used across both frontend and backend. It's like JavaScript but strictly checks for errors before the code even runs, preventing crashes.

### C. The Database and Geospatial Engine

- **Supabase:** An all-in-one backend platform that hosts the database. It acts as the central brain storing all users, parcels, and logs.
- **PostgreSQL:** The actual relational database running inside Supabase. It organizes data into tables (like rows and columns in Excel).
- **PostGIS:** A powerful extension added to PostgreSQL. This is the "secret sauce" that allows the database to understand geography. Instead of just storing numbers and text, PostGIS stores actual shapes (polygons) and can perform math on them (e.g., "Do these two shapes overlap?", "What is the area in square meters?").

### D. How They Connect (The Flow of Data)

1. **Fetching Data:** When the dashboard loads, the **Frontend (React)** asks the **Database (Supabase)** for all parcels. Supabase returns them as GeoJSON (a standard format for map data). The **Frontend** feeds this GeoJSON into **MapLibre** to draw the map.
2. **Uploading:** When an image is uploaded, the **Frontend** sends it to a **Next.js API Route**. The API Route pretends to run an AI model, generates mock polygons, and tells **Supabase** to save them in the `parcels` table.
3. **Spatial Validation:** After saving, the API tells **PostGIS** to run a function called `ST_Intersects`. PostGIS mathematically checks if the new polygons overlap with existing ones and flags them if they do.
4. **Editing:** When a user drags a point on the map, **Mapbox Draw** calculates the new coordinates. The **Frontend** sends these new coordinates to the **API Route**, which then updates the specific row in the **Supabase Database**. At the same time, it writes a log of this change for the **Model Feedback Loop**.